Gem Language Reference

David B Hon

2026-04-16

David B Hon Hard Science Gem Language Copywrite 2025, 2026

Version: 0.1.0 | **Date:** 2026-04-16 | **Built with:** Gemini CLI & Kiro CLI

Welcome to the **Gem Language** — a modern, expressive STEM language built by Gemini-CLI and Kiro-CLI using C++26 for AI, scientific computing, quantitative finance, and cross-platform application development.

Key Features

- **Explicit Typing with Initial Values**: All variable declarations require an initial value (`int x = 0`).
- **Global vs. Local Scope**: Variables starting with `_` (e.g., `_config`) are global; all others are local.
- **Universal Inheritance**: Every object inherits from `sys`, giving immediate access to `sys.print`, etc.
- **First-Class AI Support**: Built-in `ai` object supports Gemini, Mistral, and Ollama.
- **Polyglot Interop**: `use` keyword bridges Python, Julia, R, Fortran, C++, Go, Ruby, Rust, Node.js on-the-fly.
- **Mobile & Cross-Platform**: `mobl` object + browser PWA (Web Speech + Geolocation) works on Android, iPhone, macOS, Linux, and Windows 11.
- **Domain-Specific Built-ins**: `fin`, `bsm`, `bev`, `geo`, `data`, `chart`, `mobl`, `trek`, `astro`, `nlp`, `www`, `cdn`, `thread` built into the runtime.
- **Regular Expressions**: `rex` builtin provides full ECMAScript regex — match, find, findall, groups, sub, gsub, split, count.
- **Symbolic Math**: `math` builtin supports symbolic differentiation, integration, simplification, and LaTeX output via SymPy/Sage.
- **Astrophysics**: `astro` builtin covers stellar physics, orbital mechanics, cosmology, solar physics, and exoplanets.

Tutorial Roadmap

1. The Foundations

- **01_basics.g**: Variables, types, and basic I/O using the `sys` object.
- **02_math_latex.g**: Arithmetic operations and LaTeX-style mathematical expressions.
- **03_text_pdf.g**: Text manipulation and PDF document generation.
- **04_images.g**: Image processing with ImageMagick.
- **05_geo_gis.g**: GIS, geolocation, GeoJSON, and interactive maps.

2. Application Development

- **06_web_apps.g**: Building web servers with `sys.app()` and route handlers.
- **07_ai_nlp.g**: NLP and AI prompting with Gemini, Mistral, and Ollama.
- **08_functions.gm**: Defining functions, objects, and inheritance.
- **09_module_demo.g**: Structuring code with modules.

Gem v0.1.0 2026-04-16

- **11_modules_paths.g**: Managing module resolution and import paths.

3. Advanced Integration & AI

- **10_interop_cpp.g**: High-performance C++ interoperability via Cling JIT.
- **12_lib_management.g**: Dependency management and library linking.
- **13_dependency_details.g**: Deep dive into Gem's package ecosystem.
- **14_ai_providers.g**: Using Gemini, Mistral, and Ollama side-by-side.
- **15_mistral_native.g**: Native Mistral C++ bridge via `prompt_native`.

4. Specialized Tooling

- **16_algo_text_extras.g**: Advanced string algorithms and text analysis.
- **17_repl_features.g**: Mastering the interactive Gem REPL.
- **18_tcp_sockets.g**: Low-level networking fundamentals.
- **19_tcp_server.g** & **20_tcp_client.g**: Client-server architectures.
- **21_polyglot.g**: Bridging Python, Julia, and Gem.

5. Science, Finance & DevOps

- **22_builtins_itr_tcp.g**: Iterators and network protocols.
- **23_data_science.g**: DataFrames, statistics, and visualization.
- **24_devops.g**: Automation scripts and infrastructure management.
- **25_ollama_mistral.g**: Running local LLMs via Ollama.
- **26_devops_containers_vms.g**: Docker and Vagrant environments.

6. High-Performance & Symbolic Math

- **27_rust_node.g**: Polyglot support for Rust and Node.js.
- **28_interactive_charts.g**: Dynamic 2D/3D/multi-axes Plotly visualization.
- **29_quantlib_finance.g**: Quantitative finance with QuantLib.
- **30_bsm_pde_inheritance.g**: Black-Scholes-Merton PDE solvers.
- **31_history_and_compilation.g**: Gem's compilation model and history.
- **32_symbolic_math.g**: Symbolic differentiation and integration.
- **33_langport_porting.g**: Porting legacy codebases to Gem.
- **34_geo_tectonics.g**: Advanced GIS, tectonic modeling, and geo-visualization.
- **35_text_advanced.g**: Large-scale text processing and indexing.

7. Mobile & Cross-Platform

- **36_mobl_travel_log.g**: `mobl` object — NLP dictation + GPS + GeoJSON travel log PWA.

8. System Programming & Hardware

- **37_device_drivers.g**: `drvr` builtin — Linux, Windows 11, macOS, and Android device driver development.

9. Text & Pattern Matching

- **38_rex.g**: `rex` builtin — full ECMAScript regular expressions: match, find, findall, groups, sub, gsub, split, count.

10. Travel Logs & OSM Mapping

- **39_trek.g**: `trek` builtin — create, display, and edit travel logs on OpenStreetMap with GeoJSON/GPX support.

Gem Language v0.1.0 2026-04-16

11. Search Engine Optimization

- **40_seo.g**: `seo` builtin — crawl URLs, extract SEO signals, write JSON index, analyze report.

Help & Builtins

```
--- Gem Language Help ---
Gem is a high-performance STEM language implemented in C++26.

Usage Tips:
  - All variables must be initialized: 'int x = 0'
  - Global variables start with underscore: '_config = 1'
  - Use 'use "file.py"' to translate and run foreign code.
  - Use 'langport("*.py", "out.gm")' to port and save code.
  - Use 'lib' to see all loaded modules.
  - Use 'his' to view today's session history.

CLI Options:
  gem <file.g>              - Run a script.
  gem -c <main.g> [mod...]  - Compile multiple files into a binary.
  gem -o <name>             - Specify output name for compiler.
  gem -h                    - Print history and exit.
  gem -t <file> [-o output] - AI-assisted translation to Gem.

Available Builtin Modules:
  sys, math, ai, text, rex, algo, bev, file, zip, nlp, img, www, cdn, geo,
  cpp, tcp, itr, thread, data, k3s, vm, go, ruby, node, rust,
  fin, bsm, chart, astro, mobl, trek, seo, drvr

Keywords for Documentation:
  fun, obj, use, alias, his, lib, end, if, while,
  int, double, string, bool, exit

Mobile & Cross-Platform:
  mobl phone("name")  phone.dictate(text)  phone.make_feature(lat,lon,text)
  ./gem travel_log.g  ->  http://localhost:8080  (Android/iPhone/macOS/Linux/Win11)

Detailed help: help "topic"  or  help(topic)
```

Builtin Module Reference

sys — System Root Module

Every object inherits from `sys`. Methods:

- `sys.print(...)` — print to stdout
- `sys.args()` — command-line arguments as a vector
- `sys.async(func, [params], [timeout])` — run function in background thread
- `sys.sethandler(signals, func)` — set custom signal handler
- `sys.host()` — OS information string
- `sys.exec(cmd)` — execute a shell command
- `sys.doc([path])` — read file comments or current script comments
- `sys.help([topic])` — interactive help

Gem v0.1.0 · 2026-04-16

- `sys.exit()` — exit the REPL or script
- `sys.langport(pattern, [output])` — port foreign code (e.g. `*.py`) to Gem
- `sys.redirect(url, [port])` — HTTP redirect or background redirect server
- `sys.app(port, [routes])` — background web app server; routes map paths to strings, files, or handler functions

Comparisons: C++= extttstd::cout/thread/signal extperiodcentered Python= extttprint/threading/signal extperiodcentered Julia= extttprintln/@async extperiodcentered Go= extttfmt.Println/goroutine extperiodcentered Ruby= extttputs/Thread.new extperiodcentered Rust= extttprintln!/thread::spawn

math — Mathematics & Symbolic Math

Standard functions: `sin(x)`, `cos(x)`, `sqrt(x)`, constant `math.pi`.

Symbolic math (SymPy/Sage backend):

- `math.useSymPy()`, `math.useSage()` — switch symbolic provider
- `math.simplify(expr)` — symbolic simplification
- `math.diff(expr, [var])` — symbolic differentiation
- `math.integrate(expr, [var])` — symbolic integration
- `math.solve(expr, [var])` — solve algebraic equation
- `math.sym_latex(expr)` — LaTeX for a symbolic expression
- `math.to_latex(val)` — convert value or matrix to LaTeX string
- `math.write_latex(path, content, [standalone])` — write LaTeX to file
- `math.read_latex(path)` — read content from LaTeX document environment
- `math.parse_latex(text)` — extract math expressions from text (`$...$`)
- `math.compile_latex(path)` — run pdflatex on a file

Comparisons: C++= extttt<cmath>/GiNaC extperiodcentered Python= extttmath/sympy extperiodcentered Julia= extttsin()/Symbolics.jl extperiodcentered Go= extttmath.Sin() extperiodcentered Ruby= extttMath::sin() extperiodcentered Rust= extttf64::sin()

ai — AI Integration

Supports Gemini (default), Mistral, and Ollama:

- `ai.prompt(text)` — send prompt to current provider
- `ai.prompt_native(text)` — high-speed native Mistral C++ bridge (requires `mistralai` >= 1.0, Python 3.14 embedded)
- `ai.useMistral(model)`, `ai.useOllama(model, [host])`, `ai.useGemini()` — switch provider
- `ai.setKey(key)`, `ai.setHost(host)`, `ai.setPath(path)` — API configuration
- Properties: `provider`, `model`, `host`

Comparisons: C++= extttlibcurl+json extperiodcentered Python= extttgoogle-generativeai/mistralai extperiodcentered Julia= extttHTTP.jl extperiodcentered Go= extttgenerative-ai-go extperiodcentered Ruby= extttruby-openai extperiodcentered Rust= extttasync-openai

text — String & Document Processing

- `text.read(path)` — read file into string
- `text.sub(s, start, [len])` — substring
- `text.replace(s, old, new)` — replace all occurrences
- `text.write_pdf(path, text)` — create PDF; `write_pdf_a` for PDF/A archival
- `text.read_pdf(path)` — extract text from PDF
- `text.read_markdown(path)`, `text.write_markdown(path, text)` — Markdown
- `text.read_yaml(path)`, `text.write_yaml(path, obj)` — YAML
- `text.read_html(path)`, `text.write_html(path, text)` — HTML
- `text.read_xml(path)`, `text.write_xml(path, text)` — XML

Gem v0.1.0 2026-04-16

- `text.read_fits_header(path)` — read FITS file header as object
- `text.read_hdf_header(path)` — read HDF5 file header as object

Comparisons: C++= extttstd::string/fstream/pugixml extperiodcentered Python= extttstr/pypdf/pyyaml/lxml extperiodcentered Julia= extttString/YAML.jl extperiodcentered Go= extttstrings/encoding/xml extperiodcentered Ruby= extttString/prawn/nokogiri extperiodcentered Rust= extttString/lopdf/quick-xml

geo — GIS & Geolocation

- `geo.lookup()` — IP-based geolocation (`.lat`, `.lon`, `.city`, `.country`)
- `geo.distance(lat1, lon1, lat2, lon2)` — Haversine distance in km
- `geo.write_geojson(path, features)` — write GeoJSON FeatureCollection
- `geo.history(plate)` — geological history of a tectonic plate (AI-assisted)
- `geo.plot2d(data, [layout])` — 2D Plotly map (open-street-map / scattermapbox)
- `geo.plot3d(data, [layout])` — 3D globe visualization (orthographic scattergeo)

Comparisons: C++= extttGDAL/GEOS extperiodcentered Python= extttgeopandas/folium/geopy extperiodcentered Julia= extttGeoStats.jl extperiodcentered Go= extttpaulmach/orb extperiodcentered Ruby= extttrgeo extperiodcentered Rust= extttgeo crate

astro — Astrophysics & Planetary Science

Constants: `G`, `c`, `AU`, `pc`, `ly`, `Msun`, `Rsun`, `Lsun`, `Tsun`, `Mearth`, `Rearth`, `H0`, `sigma_sb`

Unit conversions: `to_ly(pc)`, `to_pc(ly)`, `to_au(m)`, `deg_to_rad(d)`, `rad_to_deg(r)`

Stellar: `luminosity(R,T)`, `stefan_boltzmann(T,R)`, `wien(T)`, `abs_magnitude(m,d_pc)`, `distance_modulus(d_pc)`, `spectral_class(T)`, `schwarzschild_radius(M)`, `escape_velocity(M,R)`

Orbital: `orbital_period(a_au,M_msun)`, `orbital_velocity(a_m,M_kg)`, `hill_sphere(a,mp,ms)`, `roche_limit(R,rho_p,rho_s)`, `synodic_period(p1,p2)`, `planet(name)` → Mercury…Neptune

Solar: `solar_flux(dist_au)`, `solar_wind_pressure(n_cm3,v_km_s)`, `sunspot_cycle(year)`, `parker_spiral_angle(v_sw` `solar_activity()`

Cosmology: `hubble_distance(z)`, `redshift_velocity(z)`, `lookback_time(z)`, `critical_density()`

Coordinates: `equatorial_to_galactic(ra,dec)`, `angular_separation(ra1,dec1,ra2,dec2)`

Exoplanet: `transit_depth(rp_rearth,rs_rsun)`, `habitable_zone(L_lsun)`, `equilibrium_temp(L,d,A)`

Pulsar: `pulsar_spindown(P_s, Pdot)` → `{age_yr, Bfield_G, edot_W}`

seo — Search Engine Optimization

- `seo.index(urls, output_path)` — crawl list of URLs, extract SEO signals, write JSON index
- `seo.analyze(index_path)` — load index JSON and print SEO report to stdout

Signals extracted: `title`, `description`, `keywords`, `og:title`, `og:description`, `canonical`, `word_count`, `img_count`, `img_with_alt`, `internal_links`, `external_links`, `h1`/`h2`/`h3` headings

Comparisons: C++=`libcurl+custom parser` · Python=`scrapy/beautifulsoup4` · Julia=`HTTP.jl/Gumbo.jl` · Go=`colly/goquery` · Ruby=`nokogiri/mechanize` · Rust=`scraper/reqwest`

trek — Travel Logs & OSM Mapping

- `trek.new(path)` — create a new empty GeoJSON travel log file
- `trek.add(path, lat, lon, [title], [note])` — append a waypoint
- `trek.edit(path, index, title, note)` — update title/note of waypoint at index
- `trek.remove(path, index)` — remove waypoint at index
- `trek.load(path)` — return GeoJSON string

Gem v0.1.0 · 2026-04-16

- `trek.show(path, [port])` — serve interactive OSM (Leaflet) map with live edit UI (default port 8090)
- `trek.export_gpx(geojson_path, gpx_path)` — export waypoints to GPX format
- `trek.stats(path)` — returns object with `.waypoints` count and `.distance_km` total

Comparisons: C++= extttGDAL/libgpx extperiodcentered Python= extttgpxpy/folium extperiodcentered Julia= extttGPX.jl extperiodcentered Go= extttkrajina/gpxgo extperiodcentered Ruby= extttgpx gem extperiodcentered Rust= extttgpx crate

Map tiles: OpenStreetMap (Leaflet.js). Waypoints stored as GeoJSON FeatureCollection. Edits persist to file.

mobl — Mobile & Cross-Platform PWA

```
mobl phone("device_name")
phone.dictate(spoken_text)         # NLP parse -> JSON {title, note, tags}
phone.make_feature(lat, lon, text) # GPS + dictation -> GeoJSON Feature
```

Browser supplies GPS (Geolocation API) + speech (Web Speech API). Gem server handles NLP, GeoJSON assembly, and file persistence.

Routes: `/` → PWA HTML, `/log` → POST `{lat,lon,text}`, `/data` → GET GeoJSON FeatureCollection

fin — Financial Engineering

- `fin.ticker(symbol)` — real-time market data via yfinance
- `fin.high_yield_bonds()`, `fin.high_yield_etfs()`, `fin.high_yield_equities()` — top 50 from TradingView
- `fin.bs_price(type, strike, spot, rate, vol, t)` — European option pricing
- `fin.greeks(type, strike, spot, rate, vol, t)` → NPV, Delta, Gamma, Theta, Vega
- `fin.date(d,m,y)`, `fin.calendar(name)`, `fin.is_holiday(cal,d,m,y)`, `fin.add_days(...)`, `fin.diff_days(...)`

Comparisons: C++= extttQuantLib extperiodcentered Python= extttyfinance/quantlib-python extperiodcentered Julia= extttMarketData.jl extperiodcentered Go= extttpiquette/quantlib extperiodcentered Ruby= extttquantlib-ruby extperiodcentered Rust= extttta crate

Comparisons: C++= extttQt extperiodcentered Python= extttFlask+SpeechRecognition extperiodcentered Julia=no built-in extperiodcentered Go= extttnet/http+custom JS extperiodcentered Ruby= extttRails+Web Speech extperiodcentered Rust= extttactix-web+custom JS

bsm — Black-Scholes-Merton

Inherits all `fin` methods, plus:

- `bsm.price_american(symbol, type, duration)` — PDE-enhanced American option pricing (`weekly/monthly/quarterly`)

Comparisons: C++= extttQuantLib FDBlackScholesVanillaEngine extperiodcentered Python= extttquantlib-python AmericanOption extperiodcentered Julia= extttQuantLib.jl AmericanExercise

chart — Interactive Plotting (Plotly.js)

- `chart.plot(traces, [layout])` — generate interactive HTML chart
- `chart.show(path)` — open chart in browser
- `chart.server(path, [port])` — serve chart via background HTTP server

Comparisons: C++= extttmatplotlib-cpp/ROOT extperiodcentered Python= extttplotly/matplotlib/seaborn extperiodcentered Julia= extttPlots.jl/Makie.jl extperiodcentered Go= extttgonum/plot extperiodcentered Ruby= extttgruff extperiodcentered Rust= extttplotters crate

Gem v0.1.0 2026-04-16

data — Data Science

- `data.read_csv(path)` — read CSV into vector of row objects
- `data.mean(vector)` — arithmetic mean
- `data.std(vector)` — sample standard deviation

Comparisons: C++= extttEigen/Armadillo extperiodcentered Python= extttpandas/numpy extperiodcentered Julia= extttDataFrames.jl/CSV.jl extperiodcentered Go= extttgonum/stat extperiodcentered Ruby= extttdaru gem extperiodcentered Rust= extttpolars crate

rex — Regular Expressions (ECMAScript)

All functions accept optional flags string (`"i"` = case-insensitive):

- `rex.match(text, pattern, [flags])` → bool
- `rex.find(text, pattern, [flags])` → first match string
- `rex.findall(text, pattern, [flags])` → list of all matches
- `rex.groups(text, pattern, [flags])` → capture groups from first match
- `rex.sub(text, pattern, repl, [flags])` → replace first match
- `rex.gsub(text, pattern, repl, [flags])` → replace all matches
- `rex.split(text, pattern, [flags])` → split on pattern
- `rex.count(text, pattern, [flags])` → count non-overlapping matches

Comparisons: C++= extttstd::regex extperiodcentered Python= extttre extperiodcentered Julia= extttRegex/match() extperiodcentered Go= extttregexp extperiodcentered Ruby= exttt= extasciitilde/gsub extperiodcentered Rust= extttregex crate

algo — Algorithms & Time

- `algo.add(...)` — sum all numeric arguments
- `algo.quicksort(v)`, `algo.sort(v, [start], [end])` — sort numeric vectors
- `algo.now()` — current local time as string
- `algo.date_add(ts, days)`, `algo.date_diff(t1, t2)` — date arithmetic

Comparisons: C++= extttstd::sort/std::chrono extperiodcentered Python= extttsorted()/datetime extperiodcentered Julia= extttsort!()/Dates extperiodcentered Go= extttsort.Slice/time extperiodcentered Ruby= extttArray#sort!/Time extperiodcentered Rust= extttslice.sort/SystemTime

tcp — TCP/IP Networking

- `tcp.listen(port)`, `tcp.accept(fd)`, `tcp.connect(host, port)` — socket management
- `tcp.send(fd, msg)`, `tcp.recv(fd, [size])`, `tcp.close(fd)` — data transfer
- `tcp.nic()` — network interface configurations
- `tcp.routes()` — system routing table

Comparisons: C++= extttPOSIX socket/bind/listen extperiodcentered Python= extttsocket.socket extperiodcentered Julia= extttSockets.listen extperiodcentered Go= extttnet.Listen extperiodcentered Ruby= extttTCPServer extperiodcentered Rust= extttTcpListener::bind

thread — Background Threads

Returned by `sys.async()`. Methods on the thread handle:

- `thread.wait()` — block until complete, returns result (or timeout error string)
- `thread.is_finished()` — bool, non-blocking status check

Comparisons: C++= extttstd::future/std::async extperiodcentered Python= extttconcurrent.futures.Future extperiodcentered Julia= exttt@async/fetch() extperiodcentered Go= extttgoroutine+channel extperiodcentered Ruby= extttThread#join/alive? extperiodcentered Rust= extttJoinHandle#join

Gem v0.1.0 · 2026-04-16

www — Web Framework

- `www.wget(url, file)` — download file via curl
- `www.app(port, [routes])` — background HTTP server; routes map paths to HTML strings
- `www.redirect(target, [port])` — start redirect server or return redirect header string
- `www.map2d(geojson, output)` — render 2D map via Mapnik (requires HAS_MAPNIK)

Comparisons: C++= extttcpp-httplib/Crow extperiodcentered Python= extttFlask/FastAPI extperiodcentered Julia= extttHTTP.jl/Genie.jl extperiodcentered Go= extttnet/http/Gin extperiodcentered Ruby= extttSinatra/Rails extperiodcentered Rust= extttactix-web/axum

cdn — Caching Proxy Server

Inherits all `www` methods, plus:

- `cdn.start(port, routes_map)` — start caching reverse-proxy; routes map path prefixes to origin URLs
- `cdn.stats()` — returns object with `.hits`, `.misses`, `.bytes`, `.cached_items`
- `cdn.purge([path])` — evict one path or `"*"` to clear entire cache
- `cdn.config(type, content)` — generate nginx/apache config

Comparisons: C++= extttnginx/Varnish extperiodcentered Python=no built-in extperiodcentered Julia=no built-in extperiodcentered Go= extttgroupcache extperiodcentered Ruby= extttrack-cache extperiodcentered Rust=no built-in

k3s — Docker & Kubernetes

- `k3s.docker_run(image, cmd)`, `k3s.docker_ps()`, `k3s.docker_build(path, tag)`, `k3s.docker_stop(id)`
- `k3s.k3s_apply(yaml)`, `k3s.k3s_get(resource)`, `k3s.k3s_pods()`, `k3s.k3s_nodes()`, `k3s.k3s_logs(pod)`

Comparisons: C++= extttDocker SDK/HTTP API extperiodcentered Python= extttdocker-py/kubernetes extperiodcentered Julia=no official SDK extperiodcentered Go= extttdocker/client/k8s.io/client-go extperiodcentered Ruby= extttdocker-api/kubeclient extperiodcentered Rust= extttbollard/kube-rs

vm — Vagrant VMs

`vm.init(box)`, `vm.up()`, `vm.ssh(cmd)`, `vm.status()`, `vm.halt()`, `vm.destroy()`

drvr — Device Driver Development

- `drvr.linux(name)`, `drvr.win11(name)`, `drvr.macos(name)`, `drvr.android(name)` — generate driver templates
- `drvr.build(target)`, `drvr.deploy(target)` — cross-platform build and deploy

Comparisons: C++= extttLinux module_init/WDK/WDF extperiodcentered Python= extttctypes/cffi (userspace only) extperiodcentered Julia=no support extperiodcentered Go= extttTinyGo extperiodcentered Ruby=no support extperiodcentered Rust= extttlinux-kernel-module-rust

Comparisons: C++= extttlibvirt C API extperiodcentered Python= extttpython-vagrant extperiodcentered Julia=no official bindings extperiodcentered Go= extttos/exec extperiodcentered Ruby=Vagrant is Ruby extperiodcentered Rust= extttstd::process::Command

Other Builtins

- **nlp** — Natural Language Processing
- **bev** — Bevington data reduction: `data(x,y)`, `fit_line()`, `param(idx)`
- **file** — `file.write(path, content)`, `file.exists(path)`
- **zip** — `zip.compress(src, archive)`, `zip.decompress(archive, dest)`
- **img** — `img.resize(src, width, height, dest)`

Gem v0.1.0 2026-04-16

- **cpp** — `cpp.repl()`, `cpp.exec(code)` — C++26 JIT via Cling
- **itr** — `itr.range(n)`, `itr.while(cond_fun, body_fun)`
- **go**, **ruby**, **node**, **rust** — polyglot `run(input)` + language-specific helpers

---

Mobile Travel Log App

```
./gem travel_log.g          # start server on :8080
# Android/iPhone → http://<host-ip>:8080
# macOS/Linux/Win11 → http://localhost:8080
```

Uses the `mobl` builtin + `travel_log.html` (PWA). Voice dictation → AI NLP → GPS pin → live GeoJSON → Plotly map.

Getting Started

Prerequisites

- **macOS/Linux**: `g++` (supporting C++2b/C++26), `make`, `python3` (3.14+), `libreadline-dev`. Python 3.14 headers required for embedded polyglot runtime.
- **Windows 11**: MSYS2 with `mingw-w64-x86_64-gcc`, `make`, and `mingw-w64-x86_64-python`. Ensure `C:\msys64\mingw64\bin` is in your PATH.

Build and Run

```
make all
./gem tutorial/01_basics.g
```

On Windows, the binaries will be produced as `gem.exe` and `gem_test.exe`.

Windows Support

Gem is now fully compatible with Windows 11 via MSYS2/MinGW-w64. Key improvements include: - **Robust Dependency Detection**: Automatic detection of Python headers and libraries using `sysconfig`. - **Modern API Integration**: Support for `cpp-httplib` and other libraries through `_WIN32_WINNT=0x0A00`. - **Cross-Platform Signals**: Graceful handling of platform-specific signals (`SIGINT`, `SIGTERM`). - **Path Compatibility**: Seamless handling of Windows-style paths and `.exe` extensions in compilation mode.

Or enter the REPL:

```
./gem
```

Developed with Gemini CLI and Kiro.

Gem v0.1.0 · 2026-04-16